Lesson 67: Recurrent Neural Networks and Backpropagation Through Time

AI/ML Mathematics

Contents

1 Motivation and Overview 2

2 References 3

3 Sequence Modeling 3

4 Recurrent Neural Network Architecture 4

5 Loss Functions for Sequences 5

6 Backpropagation Through Time 5

7 The Vanishing and Exploding Gradient Problem 7

8 Mitigation Strategies 9

9 Exercises 10

10 Summary 11

11 Further Reading 11

1 Motivation and Overview

Consider an RL agent navigating a partially observable environment. At each time step, the agent receives an observation $\mathbf{x}_t$ that depends on the full history of states and actions. A feedforward network maps each observation independently to an action, discarding all temporal structure. Yet the optimal policy often depends on the *sequence* of past observations—for instance, distinguishing whether a partially visible object is moving left or right requires comparing the current frame with previous ones. How can a neural network process variable-length sequences while maintaining a fixed-size parameter set?

Recurrent Neural Networks (RNNs) answer this question by introducing a *hidden state* $\mathbf{h}_t$ that is updated at each time step as a function of the current input and the previous hidden state. This recurrence compresses the entire history into a fixed-dimensional vector, enabling the network to model temporal dependencies. The learning algorithm that trains RNNs—Backpropagation Through Time (BPTT)—unfolds the recurrence into a deep feedforward graph and applies the chain rule. However, the repeated multiplication of Jacobian matrices along this unfolded graph leads to a fundamental pathology: gradients either *vanish* or *explode* exponentially with the sequence length. The tools developed in this lesson will let us formalize the RNN architecture, derive the BPTT algorithm, and prove sharp bounds on gradient decay and growth.

Why RNNs and BPTT Matter for RL/ML

- **Partial observability in RL.** When the agent cannot observe the full state, recurrent policies (e.g., LSTM-based actors in PPO) use hidden states to approximate belief states over the underlying MDP.
- **Sequence prediction.** Language modeling, speech recognition, and time-series forecasting all require processing variable-length sequences—the core capability of RNNs.
- **Gradient pathology.** The vanishing/exploding gradient problem discovered in vanilla RNNs motivates gated architectures (LSTM, GRU) studied in Lessons 68–69, and also informs gradient clipping strategies used throughout deep RL.
- **Credit assignment over time.** BPTT is the temporal analogue of spatial backpropagation; understanding its limitations clarifies why long-horizon credit assignment remains challenging in deep RL.

Prerequisites. Feedforward neural networks and backpropagation (Lesson 63, Section 3). Matrix norms, spectral norm, and eigenvalues (Lesson 8, Definitions 3.1–3.3). Chain rule for vector-valued functions and Jacobian matrices (Lesson 3, Theorem 2.3; Lesson 8, Section 5). Sequences and convergence (Lesson 1, Section 4).

Learning objectives. After completing this lesson, you will be able to:

- **Define** the vanilla RNN architecture and **derive** the forward equations for hidden state updates, output computation, and sequence loss.
- **Derive** the complete BPTT algorithm, including the hidden state error recurrence and weight gradient formulas.
- **Prove** that gradients in vanilla RNNs decay exponentially when the spectral norm $\|\mathbf{U}\| < 1$ (vanishing gradient theorem).
- **Prove** that gradients can grow exponentially when $\|\mathbf{U}\| > 1$ (exploding gradient theorem).
- **Analyze** the eigenvalue structure of the recurrent weight matrix and its role in determining gradient behavior.

- **Explain** gradient clipping, orthogonal initialization, and truncated BPTT as mitigation strategies, and **evaluate** their limitations.

2 References

- Goodfellow et al. [2016, Chapter 10] — Comprehensive treatment of RNN architectures, BPTT, and gradient flow analysis.
- Elman [1990] — Original Elman network introducing the simple recurrent architecture with hidden state feedback.
- Rumelhart et al. [1986] — Backpropagation algorithm for feedforward networks; the foundation on which BPTT builds.
- Bengio et al. [1994] — Seminal analysis of why learning long-term dependencies with gradient descent is difficult.
- Pascanu et al. [2013] — Detailed analysis of the vanishing and exploding gradient problem with gradient clipping solutions.
- Hochreiter [1991] — Early identification of the vanishing gradient problem in recurrent networks.

3 Sequence Modeling

Definition 3.1 (Sequence)**.** A *sequence* of length T over a set $\mathcal{X}$ is an ordered tuple

$$\mathbf{x}_{1:T} = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T), \quad \mathbf{x}_t \in \mathcal{X} \text{ for all } t \in \{1, \ldots, T\}.$$

The fundamental challenge in sequence modeling is to learn a function that captures dependencies between elements at different positions. Feedforward networks treat each input independently, discarding positional information unless it is explicitly encoded.

Definition 3.2 (Sequence-to-Sequence Map)**.** A *sequence-to-sequence map* is a function $F : \mathcal{X}^T \to \mathcal{Y}^T$ that maps an input sequence $\mathbf{x}_{1:T}$ to an output sequence $\mathbf{y}_{1:T} = (F(\mathbf{x}_{1:T})_1, \ldots, F(\mathbf{x}_{1:T})_T)$. The map is *causal* if each output $\mathbf{y}_t$ depends only on current and past inputs $\mathbf{x}_1, \ldots, \mathbf{x}_t$.

Example 3.3 (Language Modeling)**.** In language modeling, $\mathbf{x}_t$ is the one-hot encoding of the t-th word and $\mathbf{y}_t = P(\mathbf{x}_{t+1} \mid \mathbf{x}_{1:t})$ is the conditional distribution over the next word. This is a causal sequence-to-sequence map: predicting the next word requires only the past context.

Sequences in RL

In reinforcement learning, the agent's trajectory $(\mathbf{o}_1, a_1, r_1, \mathbf{o}_2, a_2, r_2, \ldots)$ is a sequence. Under partial observability (POMDPs), the optimal policy π^* may depend on the entire observation history $\mathbf{o}_{1:t}$, not just the current observation $\mathbf{o}_t$. An RNN can approximate this by encoding the history into a hidden state $\mathbf{h}_t$ that serves as a surrogate for the belief state.

We now introduce the recurrence principle that allows a neural network to process sequences with a fixed number of parameters.

4 Recurrent Neural Network Architecture

Definition 4.1 (Recurrence Relation)**.** A *recurrence relation* defines a hidden state $\mathbf{h}_t$ as a function of the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \boldsymbol{\theta}),$$

where f is a parameterized function and $\boldsymbol{\theta}$ represents the learnable parameters.

Definition 4.2 (Vanilla RNN)**.** A *vanilla recurrent neural network* (Elman network) with input dimension n, hidden dimension m, and output dimension k is defined by the following equations. At each time step $t = 1, \ldots, T$:

$$\mathbf{a}_t = \mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}, \tag{1}$$
$$\mathbf{h}_t = \tanh(\mathbf{a}_t), \tag{2}$$
$$\mathbf{o}_t = \mathbf{V}\mathbf{h}_t + \mathbf{c}, \tag{3}$$
$$\mathbf{y}_t = g(\mathbf{o}_t), \tag{4}$$

where:

- $\mathbf{W} \in \mathbb{R}^{m\times n}$ is the input-to-hidden weight matrix,
- $\mathbf{U} \in \mathbb{R}^{m\times m}$ is the hidden-to-hidden (recurrent) weight matrix,
- $\mathbf{V} \in \mathbb{R}^{k\times m}$ is the hidden-to-output weight matrix,
- $\mathbf{b} \in \mathbb{R}^m$ and $\mathbf{c} \in \mathbb{R}^k$ are bias vectors,
- g is an output activation function (identity for regression, softmax for classification),
- $\mathbf{h}_0 = \mathbf{0} \in \mathbb{R}^m$ (or learned).

RNN Forward Equations

The complete RNN forward pass is:

$$\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}), \tag{5}$$
$$\mathbf{y}_t = g(\mathbf{V}\mathbf{h}_t + \mathbf{c}), \tag{6}$$

with initial condition $\mathbf{h}_0 = \mathbf{0} \in \mathbb{R}^m$.

Remark 4.3 (Parameter Sharing). The same parameters $(\mathbf{W}, \mathbf{U}, \mathbf{V}, \mathbf{b}, \mathbf{c})$ are used at every time step. This provides: (i) the ability to process sequences of any length, (ii) a parameter count independent of sequence length, and (iii) statistical sharing of patterns across positions.

Proposition 4.4 (Parameter Count)**.** *For a vanilla RNN with input dimension n, hidden dimension m, and output dimension k, the total number of trainable parameters is:*

$$P = m(n + m + 1) + k(m + 1).$$

Proof. The parameters are: $\mathbf{W} \in \mathbb{R}^{m\times n}$ (mn entries), $\mathbf{U} \in \mathbb{R}^{m\times m}$ (m^2 entries), $\mathbf{b} \in \mathbb{R}^m$ (m entries), $\mathbf{V} \in \mathbb{R}^{k\times m}$ (km entries), $\mathbf{c} \in \mathbb{R}^k$ (k entries). Total: $mn + m^2 + m + km + k = m(n + m + 1) + k(m + 1)$. □

Definition 4.5 (Unfolded RNN)**.** For a sequence of length T, the *unfolded* RNN is the composition:

$$\mathbf{h}_T = f(f(\cdots f(f(\mathbf{h}_0, \mathbf{x}_1), \mathbf{x}_2) \cdots, \mathbf{x}_{T-1}), \mathbf{x}_T),$$

where $f(\mathbf{h}, \mathbf{x}) = \tanh(\mathbf{W}\mathbf{x} + \mathbf{U}\mathbf{h} + \mathbf{b})$. This can be viewed as a deep feedforward network with T layers, all sharing the same parameters.

Example 4.6 (Simple RNN Computation)**.** Consider a scalar RNN with $n = m = k = 1$, weights $W = 0.5$, $U = 0.8$, $V = 1$, biases $b = c = 0$, and inputs $x_1 = 1, x_2 = 0, x_3 = -1$. Starting from $h_0 = 0$:

$$\begin{aligned} h_1 &= \tanh(0.5 \cdot 1 + 0.8 \cdot 0) = \tanh(0.5) \approx 0.462, \\ h_2 &= \tanh(0.5 \cdot 0 + 0.8 \cdot 0.462) = \tanh(0.370) \approx 0.354, \\ h_3 &= \tanh(0.5 \cdot (-1) + 0.8 \cdot 0.354) = \tanh(-0.217) \approx -0.214. \end{aligned}$$

The hidden state retains a trace of past inputs: h_3 is influenced by all three inputs despite receiving only x_3 directly.

The RNN architecture defines a causal sequence-to-sequence map. We now define the loss function used to train it.

5 Loss Functions for Sequences

Definition 5.1 (Sequence Loss)**.** For a sequence with target outputs $\hat{\mathbf{y}}_{1:T}$, the *total sequence loss* is:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{t=1}^{T} \mathcal{L}_t(\mathbf{y}_t, \hat{\mathbf{y}}_t), \tag{7}$$

where $\mathcal{L}_t$ is the per-timestep loss function.

Definition 5.2 (Common Loss Functions)**.** For regression tasks with continuous targets:

$$\mathcal{L}_t(\mathbf{y}_t, \hat{\mathbf{y}}_t) = \frac{1}{2}\|\mathbf{y}_t - \hat{\mathbf{y}}_t\|_2^2 \quad \text{(mean squared error)}.$$

For classification with softmax outputs and one-hot targets:

$$\mathcal{L}_t(\mathbf{y}_t, \hat{\mathbf{y}}_t) = -\sum_{i=1}^{k} \hat{y}_t^{(i)} \log y_t^{(i)} = -\log y_t^{(c_t)} \quad \text{(cross-entropy)},$$

where c_t is the true class index.

Proposition 5.3 (Softmax–Cross-Entropy Gradient)**.** *When using softmax output with cross-entropy loss, the gradient of $\mathcal{L}_t$ with respect to the pre-activation $\mathbf{o}_t$ simplifies to:*

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{o}_t} = \mathbf{y}_t - \hat{\mathbf{y}}_t. \tag{8}$$

Proof. For the j-th component with true class c_t:

$$\frac{\partial \mathcal{L}_t}{\partial o_t^{(j)}} = -\frac{\partial}{\partial o_t^{(j)}} \log \frac{e^{o_t^{(c_t)}}}{\sum_i e^{o_t^{(i)}}} = -\delta_{jc_t} + \frac{e^{o_t^{(j)}}}{\sum_i e^{o_t^{(i)}}} = y_t^{(j)} - \hat{y}_t^{(j)}. \qquad \square$$

With the forward pass and loss defined, we derive the algorithm for computing gradients.

6 Backpropagation Through Time

Backpropagation Through Time (BPTT) computes gradients of the sequence loss $\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t$ by applying the chain rule to the unfolded computational graph. The key insight is that $\mathbf{h}_t$ influences the loss both directly (through $\mathcal{L}_t$) and indirectly (through all future losses $\mathcal{L}_{t+1}, \ldots, \mathcal{L}_T$ via the recurrence).

Definition 6.1 (Output Error Signal)**.** The *output error signal* at time t is:

$$\boldsymbol{\delta}_t^{(o)} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{o}_t} \in \mathbb{R}^k. \tag{9}$$

For softmax with cross-entropy: $\boldsymbol{\delta}_t^{(o)} = \mathbf{y}_t - \hat{\mathbf{y}}_t$.

Definition 6.2 (Hidden State Error Signal)**.** The *hidden state error signal* at time t accumulates contributions from all future losses:

$$\boldsymbol{\delta}_t^{(h)} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \in \mathbb{R}^m. \tag{10}$$

Theorem 6.3 (Hidden State Error Recurrence)**.** *The hidden state error signal satisfies the backward recurrence:*

$$\boldsymbol{\delta}_t^{(h)} = \mathbf{V}^\top \boldsymbol{\delta}_t^{(o)} + \mathbf{U}^\top \big(\boldsymbol{\delta}_{t+1}^{(h)} \odot (1 - \mathbf{h}_{t+1}^2)\big) \cdot \mathbf{1}_{t<T}, \tag{11}$$

where $\odot$ *denotes element-wise multiplication,* $\mathbf{h}_{t+1}^2$ *denotes element-wise squaring, and* $(1-\mathbf{h}_{t+1}^2)$ *is the derivative of* tanh *evaluated at* $\mathbf{a}_{t+1}$.

Proof. The hidden state $\mathbf{h}_t$ influences the total loss through two paths:

1. *Direct path* through $\mathbf{y}_t$: $\mathbf{h}_t \to \mathbf{o}_t \to \mathbf{y}_t \to \mathcal{L}_t$.
2. *Indirect path* through $\mathbf{h}_{t+1}$ (when $t < T$): $\mathbf{h}_t \to \mathbf{a}_{t+1} \to \mathbf{h}_{t+1} \to \cdots \to \mathcal{L}$.

For the direct path, since $\mathbf{o}_t = \mathbf{V}\mathbf{h}_t + \mathbf{c}$:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} = \mathbf{V}^\top \frac{\partial \mathcal{L}_t}{\partial \mathbf{o}_t} = \mathbf{V}^\top \boldsymbol{\delta}_t^{(o)}.$$

For the indirect path (when $t < T$), since $\mathbf{a}_{t+1} = \mathbf{W}\mathbf{x}_{t+1} + \mathbf{U}\mathbf{h}_t + \mathbf{b}$ and $\mathbf{h}_{t+1} = \tanh(\mathbf{a}_{t+1})$:

$$\left.\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t}\right|_{\text{indirect}} = \frac{\partial \mathbf{a}_{t+1}^\top}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_{t+1}^\top}{\partial \mathbf{a}_{t+1}} \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t+1}} = \mathbf{U}^\top \operatorname{diag}(1 - \mathbf{h}_{t+1}^2) \boldsymbol{\delta}_{t+1}^{(h)}.$$

Combining both paths yields the recurrence. □

Definition 6.4 (Pre-activation Error)**.** The error with respect to the pre-activation is:

$$\boldsymbol{\delta}_t^{(a)} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_t} = \boldsymbol{\delta}_t^{(h)} \odot (1 - \mathbf{h}_t^2). \tag{12}$$

Theorem 6.5 (BPTT Weight Gradients)**.** *The gradients with respect to the weight matrices and bias vectors are:*

$$\frac{\partial \mathcal{L}}{\partial \mathbf{V}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(o)} \mathbf{h}_t^\top, \tag{13}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(a)} \mathbf{x}_t^\top, \tag{14}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{U}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(a)} \mathbf{h}_{t-1}^\top, \tag{15}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(o)}, \tag{16}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(a)}. \tag{17}$$

Proof. For $\mathbf{V}$: at each time t, $\mathbf{o}_t = \mathbf{V}\mathbf{h}_t + \mathbf{c}$, so $\frac{\partial \mathcal{L}_t}{\partial \mathbf{V}} = \boldsymbol{\delta}_t^{(o)} \mathbf{h}_t^\top$. Summing over t gives (13).

For $\mathbf{W}$ and $\mathbf{U}$: since $\mathbf{a}_t = \mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}$, we have $\frac{\partial \mathbf{a}_t}{\partial \mathbf{W}} = \mathbf{x}_t^\top$ and $\frac{\partial \mathbf{a}_t}{\partial \mathbf{U}} = \mathbf{h}_{t-1}^\top$. The total gradient accumulates over all time steps:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \mathbf{a}_t} \frac{\partial \mathbf{a}_t}{\partial \mathbf{W}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(a)} \mathbf{x}_t^\top.$$

The derivation for $\mathbf{U}$, $\mathbf{b}$, and $\mathbf{c}$ is analogous. □

Complete BPTT Algorithm

Forward Pass:

1. Initialize $\mathbf{h}_0 = \mathbf{0}$.
2. For $t = 1$ to T: compute $\mathbf{a}_t$, $\mathbf{h}_t$, $\mathbf{o}_t$, $\mathbf{y}_t$, and $\mathcal{L}_t$.

Backward Pass:

1. Initialize $\boldsymbol{\delta}_{T+1}^{(h)} = \mathbf{0}$.
2. For $t = T$ down to 1:
   - $\boldsymbol{\delta}_t^{(o)} = \mathbf{y}_t - \hat{\mathbf{y}}_t$
   - $\boldsymbol{\delta}_t^{(h)} = \mathbf{V}^\top \boldsymbol{\delta}_t^{(o)} + \mathbf{U}^\top (\boldsymbol{\delta}_{t+1}^{(h)} \odot (1 - \mathbf{h}_{t+1}^2))$
   - $\boldsymbol{\delta}_t^{(a)} = \boldsymbol{\delta}_t^{(h)} \odot (1 - \mathbf{h}_t^2)$
   - Accumulate weight gradients.

Example 6.6 (BPTT for a Two-Step Sequence)**.** Consider $T = 2$ with $m = 2$, $k = 1$ (regression, so g is identity). The total loss is $\mathcal{L} = \mathcal{L}_1 + \mathcal{L}_2$. The backward pass computes:

$$\begin{aligned}
\boldsymbol{\delta}_2^{(h)} &= \mathbf{V}^\top \boldsymbol{\delta}_2^{(o)}, \\
\boldsymbol{\delta}_1^{(h)} &= \mathbf{V}^\top \boldsymbol{\delta}_1^{(o)} + \mathbf{U}^\top (\boldsymbol{\delta}_2^{(h)} \odot (1 - \mathbf{h}_2^2)).
\end{aligned}$$

Note that $\boldsymbol{\delta}_1^{(h)}$ receives gradient information from *both* $\mathcal{L}_1$ and $\mathcal{L}_2$, reflecting the influence of $\mathbf{h}_1$ on the entire future.

The BPTT algorithm reveals a critical structural property: the backward pass involves repeated multiplication by $\mathbf{U}^\top$ scaled by the tanh derivative. We now analyze the consequences.

7 The Vanishing and Exploding Gradient Problem

Theorem 7.1 (Long-Range Gradient)**.** *The gradient of the loss at time t with respect to the hidden state at an earlier time $\tau < t$ is:*

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \prod_{i=\tau+1}^{t} \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}}. \tag{18}$$

Proof. By repeated application of the chain rule along the path $\mathbf{h}_\tau \to \mathbf{h}_{\tau+1} \to \cdots \to \mathbf{h}_t \to \mathcal{L}_t$:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \cdot \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \cdots \frac{\partial \mathbf{h}_{\tau+1}}{\partial \mathbf{h}_\tau} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \prod_{i=\tau+1}^{t} \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}}.$$ □

Lemma 7.2 (Jacobian of Hidden State Transition)**.** *The Jacobian of the hidden state transition is:*

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \operatorname{diag}(1 - \mathbf{h}_t^2)\, \mathbf{U}, \tag{19}$$

where $\operatorname{diag}(1 - \mathbf{h}_t^2)$ *is the diagonal matrix with entries* $(1 - h_t^{(j)2})$.

Proof. Since $\mathbf{h}_t = \tanh(\mathbf{a}_t)$ and $\mathbf{a}_t = \mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}$:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \frac{\partial \mathbf{h}_t}{\partial \mathbf{a}_t} \cdot \frac{\partial \mathbf{a}_t}{\partial \mathbf{h}_{t-1}} = \operatorname{diag}(1 - \mathbf{h}_t^2)\,\mathbf{U}. \qquad \square$$

Theorem 7.3 (Vanishing Gradient)**.** *Let $\gamma = \|\mathbf{U}\|$ be the spectral norm of $\mathbf{U}$. If $\gamma < 1$, then:*

$$\left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau} \right\| \leq \left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \right\| \gamma^{t-\tau}. \tag{20}$$

*Thus, gradients decay **exponentially** with the time gap $(t - \tau)$.*

Proof. Using the submultiplicativity of matrix norms and Theorem 7.1:

$$\left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau} \right\| = \left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \prod_{i=\tau+1}^{t} \operatorname{diag}(1 - \mathbf{h}_i^2)\,\mathbf{U} \right\| \leq \left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \right\| \prod_{i=\tau+1}^{t} \|\operatorname{diag}(1 - \mathbf{h}_i^2)\| \cdot \|\mathbf{U}\|.$$

Since tanh maps to $(-1, 1)$, we have $|1 - h_i^{(j)2}| \leq 1$ for all components, so $\|\operatorname{diag}(1 - \mathbf{h}_i^2)\| \leq 1$. Therefore:

$$\left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau} \right\| \leq \left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \right\| \|\mathbf{U}\|^{t-\tau} = \left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \right\| \gamma^{t-\tau}. \qquad \square$$

Vanishing Gradient Bound

When $\gamma = \|\mathbf{U}\| < 1$, the gradient $\|\partial \mathcal{L}_t / \partial \mathbf{h}_\tau\|$ decays at rate $\gamma^{t-\tau}$. For $\gamma = 0.9$ and a time gap of $k = t - \tau$ steps:

k	10	50	100	200
γ^k	0.35	5×10^{-3}	2.7×10^{-5}	7×10^{-10}

Gradients become negligible after a few dozen time steps.

Theorem 7.4 (Exploding Gradient)**.** *If $\gamma = \|\mathbf{U}\| > 1$ and the Jacobian products do not shrink sufficiently (i.e., the activation derivatives remain bounded away from zero), then gradients can grow exponentially:*

$$\left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau} \right\| \geq C \cdot \gamma_{\min}^{t-\tau}, \tag{21}$$

for some constant $C > 0$, where $\gamma_{\min} > 0$ depends on the minimum activation derivative along the trajectory.

Proof. If $\sigma_{\min}(\mathbf{U}) > 0$ denotes the smallest singular value of $\mathbf{U}$ and the diagonal entries of $\operatorname{diag}(1 - \mathbf{h}_i^2)$ are bounded below by some $\alpha > 0$ (the network operates in the linear regime of tanh), then:

$$\left\| \prod_{i=\tau+1}^{t} \operatorname{diag}(1 - \mathbf{h}_i^2)\,\mathbf{U} \right\| \geq \prod_{i=\tau+1}^{t} \alpha \cdot \sigma_{\min}(\mathbf{U}) = (\alpha\, \sigma_{\min}(\mathbf{U}))^{t-\tau}.$$

When $\alpha\, \sigma_{\min}(\mathbf{U}) > 1$, the product grows exponentially. In practice, even $\|\mathbf{U}\| > 1$ often suffices since the activation derivatives are not uniformly small. $\square$

The Gradient Dilemma

The vanishing and exploding gradient problems create a fundamental dilemma for vanilla RNNs:

- If $\|\mathbf{U}\| < 1$: long-range dependencies cannot be learned (gradients vanish).

- If $\|\mathbf{U}\| > 1$: training becomes unstable (gradients explode).
- If $\|\mathbf{U}\| = 1$: marginally stable but practically unachievable with fixed weights.

This means vanilla RNNs cannot effectively learn dependencies spanning more than approximately 10–20 time steps.

Proposition 7.5 (Eigenvalue Characterization)**.** *Let $\lambda_1, \ldots, \lambda_m$ be the eigenvalues of $\mathbf{U}$. The gradient behavior is characterized by the spectral radius $\rho(\mathbf{U}) = \max_j |\lambda_j|$:*

- *If $|\lambda_{\max}| < 1$: vanishing gradients dominate.*
- *If $|\lambda_{\max}| > 1$: exploding gradient risk.*
- *If $|\lambda_{\max}| = 1$: marginally stable (ideal but fragile).*

Example 7.6 (Vanishing Gradient Demonstration)**.** Consider $m = 2$ with $\mathbf{U} = \begin{pmatrix} 0.5 & 0.3 \\ 0.2 & 0.4 \end{pmatrix}$. The eigenvalues are $\lambda_1 \approx 0.72$ and $\lambda_2 \approx 0.18$, so $\|\mathbf{U}\| < 1$. After $k = 20$ steps, the gradient is attenuated by at most $0.72^{20} \approx 0.0013$—three orders of magnitude smaller. The network effectively cannot learn dependencies beyond ∼20 steps.

Gradient Pathology in Deep RL

In recurrent policies for POMDPs, the vanishing gradient problem limits the effective memory horizon. An LSTM-based PPO agent can maintain information over hundreds of steps (Lesson 68), while a vanilla RNN policy is limited to ∼10–20 steps. This is why virtually all modern recurrent RL architectures use gated units.

Having identified the gradient pathology, we now discuss strategies to mitigate it.

8 Mitigation Strategies

Definition 8.1 (Gradient Clipping)**.** *Gradient clipping* rescales the gradient when its norm exceeds a threshold $\theta > 0$:

$$\bar{\mathbf{g}} = \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\| \le \theta, \\ \frac{\theta}{\|\mathbf{g}\|}\mathbf{g} & \text{if } \|\mathbf{g}\| > \theta. \end{cases} \tag{22}$$

Remark 8.2 (Clipping Properties). Gradient clipping preserves the gradient direction while bounding its magnitude. It addresses the exploding gradient problem but does *not* solve the vanishing gradient problem. The threshold θ is a hyperparameter; Pascanu et al. [2013] recommend values in the range $[1, 10]$.

Proposition 8.3 (Orthogonal Initialization)**.** *If $\mathbf{U}$ is initialized as an orthogonal matrix ($\mathbf{U}^\top\mathbf{U} = \mathbf{I}$), then $\|\mathbf{U}\| = 1$ and all singular values equal 1. This provides ideal gradient flow at initialization: $\|\operatorname{diag}(1 - \mathbf{h}_i^2)\,\mathbf{U}\| \le 1$ with equality when the network operates in the linear regime.*

Proof. For an orthogonal matrix, all singular values are 1, so $\|\mathbf{U}\| = 1$. The Jacobian product satisfies:

$$\left\| \prod_{i=\tau+1}^{t} \operatorname{diag}(1 - \mathbf{h}_i^2)\,\mathbf{U} \right\| \le \prod_{i=\tau+1}^{t} \|\operatorname{diag}(1 - \mathbf{h}_i^2)\| \cdot \|\mathbf{U}\| \le 1.$$

When the pre-activations are near zero (as at initialization with small inputs), $1 - h_i^{(j)2} \approx 1$, yielding $\|\operatorname{diag}(1 - \mathbf{h}_i^2)\,\mathbf{U}\| \approx 1$. □

Orthogonality is Not Preserved

Although orthogonal initialization provides ideal gradient flow at the start of training, gradient updates break the orthogonality of $\mathbf{U}$. After several training steps, the singular values of $\mathbf{U}$ drift away from 1, and the vanishing/exploding gradient problem resurfaces.

Definition 8.4 (Truncated BPTT)**.** *Truncated BPTT* with window size k limits backpropagation to the most recent k time steps:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau} \approx \begin{cases} \text{computed} & \text{if } t-\tau \le k, \\ 0 & \text{if } t-\tau > k. \end{cases} \tag{23}$$

Remark 8.5 (Truncated BPTT Trade-off). Truncated BPTT introduces bias by ignoring long-range dependencies but reduces computational cost from $O(T)$ to $O(k)$ per step. It is a practical compromise when full BPTT is too expensive. The window size k controls the bias–computation trade-off: larger k captures longer dependencies but costs more.

Example 8.6 (Comparing Mitigation Strategies)**.** Consider training a vanilla RNN on sequences of length $T = 100$:

- *No mitigation*: gradients vanish for $t - \tau > 20$; the network can only learn short-range patterns.
- *Gradient clipping* ($\theta = 5$): prevents numerical overflow from exploding gradients but does not help with vanishing gradients.
- *Truncated BPTT* ($k = 20$): reduces computation but explicitly limits the memory horizon.
- *Orthogonal initialization*: helps at the start of training but degrades as $\mathbf{U}$ drifts.

The fundamental solution requires redesigning the architecture, which motivates gated RNNs (Lessons 68–69).

9 Exercises

Exercise 9.1 (RNN Forward Pass)**.** ($*$)
Consider a vanilla RNN with $n = 2$, $m = 3$, $k = 1$, and

$$\mathbf{W} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{U} = \begin{pmatrix} 0.5 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 0.5 \end{pmatrix}, \quad \mathbf{V} = \begin{pmatrix} 1 & 1 & 1 \end{pmatrix},$$

with zero biases and $\mathbf{h}_0 = \mathbf{0}$. Compute the hidden states $\mathbf{h}_1, \mathbf{h}_2$ and outputs y_1, y_2 for the input sequence $\mathbf{x}_1 = (1, 0)^\top$, $\mathbf{x}_2 = (0, 1)^\top$.

Exercise 9.2 (BPTT Derivation for Scalar RNN)**.** ($**$)
For a scalar RNN ($n = m = k = 1$) with loss $\mathcal{L} = \frac{1}{2}\sum_{t=1}^{T}(y_t - \hat{y}_t)^2$ and $g =$ identity, derive explicit expressions for $\frac{\partial \mathcal{L}}{\partial W}$, $\frac{\partial \mathcal{L}}{\partial U}$, and $\frac{\partial \mathcal{L}}{\partial V}$ for a sequence of length $T = 3$. Verify that the gradient with respect to U involves contributions from all time steps.

Exercise 9.3 (Gradient Norm Bound)**.** ($**$)
Prove that for the vanilla RNN with tanh activation, the gradient satisfies:

$$\left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau} \right\| \le \left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \right\| \cdot \left(\max_j |1 - h_j^2| \cdot \|\mathbf{U}\| \right)^{t-\tau}.$$

Under what conditions on the hidden state values does this become a tighter bound than the one in Theorem 7.3?

Hint: Consider what happens when the hidden state components are large in magnitude (near saturation).

Exercise 9.4 (ReLU Activation in RNNs)**.** (∗∗)
Suppose we replace tanh with $\mathrm{ReLU}(z) = \max(0, z)$ in the vanilla RNN. Derive the Jacobian $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1}$ in this case. Explain why the vanishing gradient problem persists (or worsens) and why the exploding gradient problem becomes more severe.

Exercise 9.5 (Optimal Memory Horizon)**.** (∗ ∗ ∗)
Consider a vanilla RNN trained on a task where the target at time T depends on the input at time 1 (a "copy" task with delay $T - 1$). Using the vanishing gradient bound (Theorem 7.3), derive an upper bound on the sequence length T for which the gradient signal from time 1 remains above a threshold $\epsilon > 0$. Express your answer in terms of γ and ϵ. For $\gamma = 0.95$ and $\epsilon = 10^{-6}$, what is the maximum effective delay?

Hint: Solve $\gamma^{T-1} \geq \epsilon$ for T.

10 Summary

- A recurrent neural network maintains a hidden state $\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b})$ that compresses the entire input history into a fixed-dimensional vector.
- Parameter sharing across time steps allows RNNs to process sequences of arbitrary length with $O(m(n + m + 1) + k(m + 1))$ parameters, independent of sequence length.
- BPTT computes gradients by unfolding the recurrence into a deep feedforward graph and propagating errors backward through time, yielding weight gradients that accumulate over all time steps.
- The long-range gradient involves a product of $(t - \tau)$ Jacobian matrices $\mathrm{diag}(1 - \mathbf{h}_i^2)\mathbf{U}$, leading to exponential decay or growth.
- When $\|\mathbf{U}\| < 1$, gradients vanish at rate $\gamma^{t-\tau}$, preventing the network from learning dependencies beyond ∼10–20 time steps.
- When $\|\mathbf{U}\| > 1$, gradients can explode, causing training instability and numerical overflow.
- Gradient clipping addresses exploding gradients by rescaling the gradient vector, while orthogonal initialization and truncated BPTT provide partial remedies.
- The fundamental solution to the vanishing gradient problem requires architectural innovation: gated recurrent networks (LSTM, GRU) create additive paths for gradient flow.

11 Further Reading

- Lesson 68 introduces the LSTM architecture, which solves the vanishing gradient problem through an additive cell state update and learnable gating mechanisms.
- Lesson 69 covers the GRU (a simplified gated architecture), bidirectional RNNs, deep stacked RNNs, and the encoder-decoder framework for sequence-to-sequence tasks.
- Pascanu et al. [2013] provides a detailed analysis of gradient flow in RNNs, including second-order methods and the connection between gradient clipping and trust regions.
- Bengio et al. [1994] gives the original theoretical analysis of why learning long-term dependencies is fundamentally difficult, including connections to dynamical systems theory.
- Goodfellow et al. [2016, Chapter 10] offers a comprehensive survey of recurrent architectures, training techniques, and practical considerations.

- Graves [2012] covers supervised sequence labelling with RNNs, including connectionist temporal classification (CTC) for speech and handwriting recognition.

References

Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.

Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

Alex Graves. *Supervised Sequence Labelling with Recurrent Neural Networks*. Springer, 2012.

Sepp Hochreiter. Untersuchungen zu dynamischen neuronalen netzen. Diploma thesis, Technische Universität München, 1991.

Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, pages 1310–1318, 2013.

David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.